Machine-Level Programming: Procedures

Arquitectura de Computadores
Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

Today

- Mechanisms in procedures
- Stack structure
- Passing control
- Calling conventions
- Passing data
- Managing local data

Mechanisms in procedures

- **Passing control**
  - To beginning of procedure code
  - Back to return point
- **Passing data**
  - Procedure arguments
  - Return value
- **Memory management**
  - Allocate during procedure execution
  - Deallocate upon return
- **These mechanisms are implemented with stack support**

```
void P(…)
{
   •
   •
   y = Q(x);
   print(y)
   •
}
```

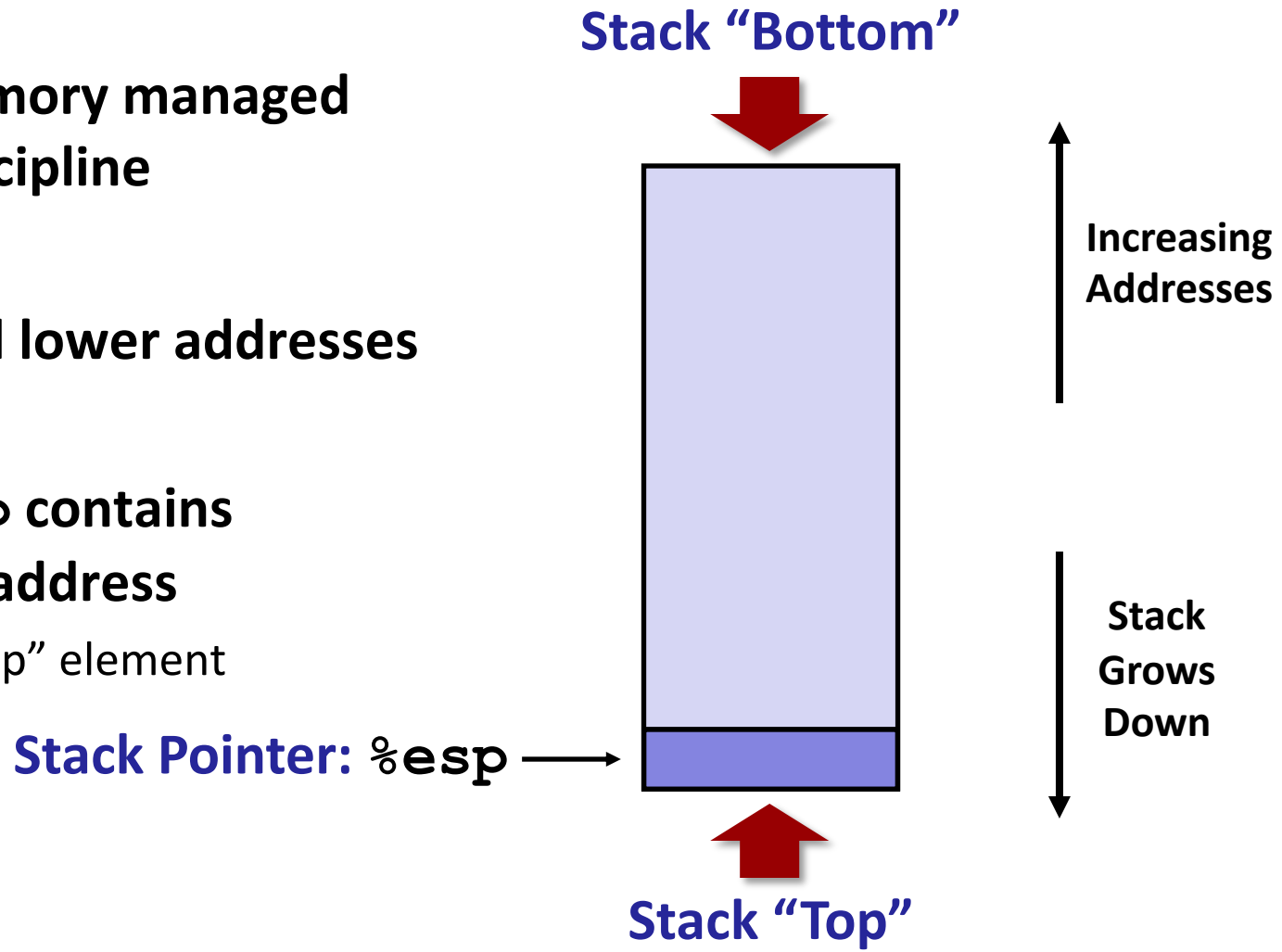
```
int Q(int i)
{
   int t = 3*i;
   int v[10];
   •
   •
   return v[t];
}
```

Today

- Mechanisms in procedures
- **Stack structure**
- Passing control
- Calling conventions
- Passing data
- Managing local data

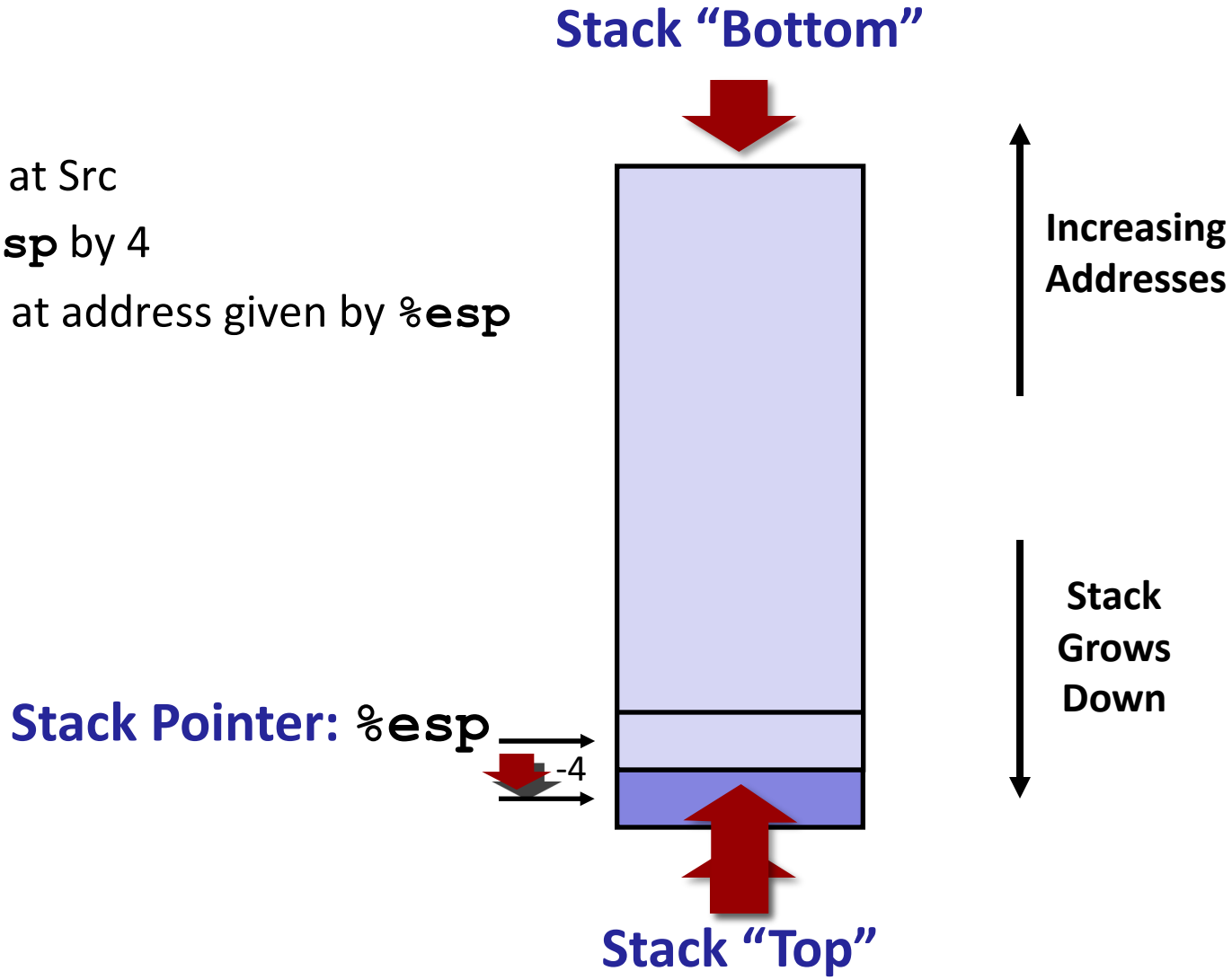
IA32 Stack

- **Region of memory managed with stack discipline**
- **Grows toward lower addresses**
- **Register `%esp` contains lowest stack address**
  - Address of "top" element



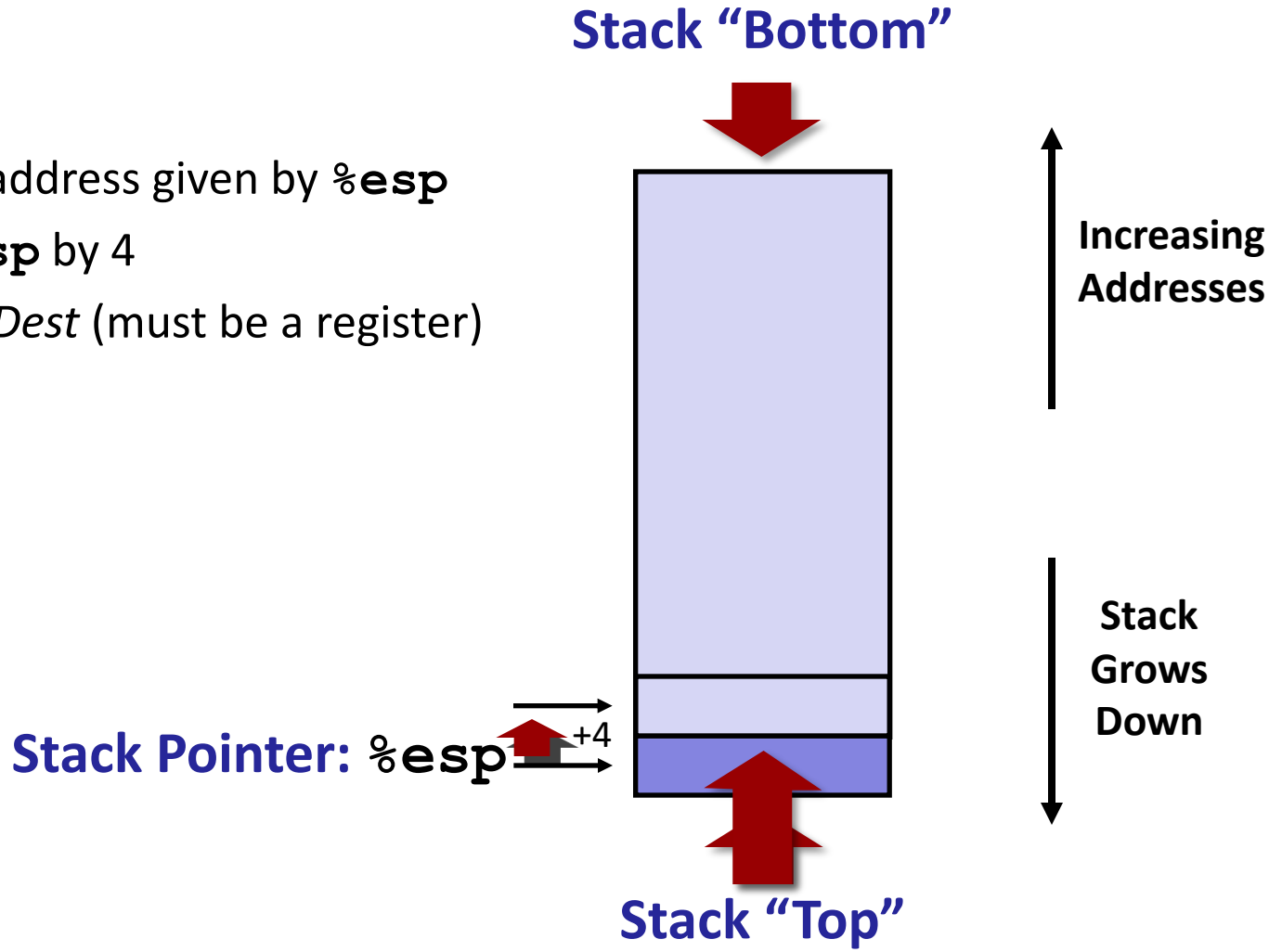
IA32 Stack: Push

- **`pushl Src`**
  - Fetch operand at Src
  - Decrement **`%esp`** by 4
  - Write operand at address given by **`%esp`**



IA32 Stack: Pop

- **`popl` Dest**
  - Read value at address given by **`%esp`**
  - Increment **`%esp`** by 4
  - Store value at *Dest* (must be a register)



Today

- Mechanisms in procedures
- Stack structure
- **Passing control**
- Passing data
- Calling conventions
- Managing local data

Procedure control flow

- **IA32 uses stack to support procedure call and return**

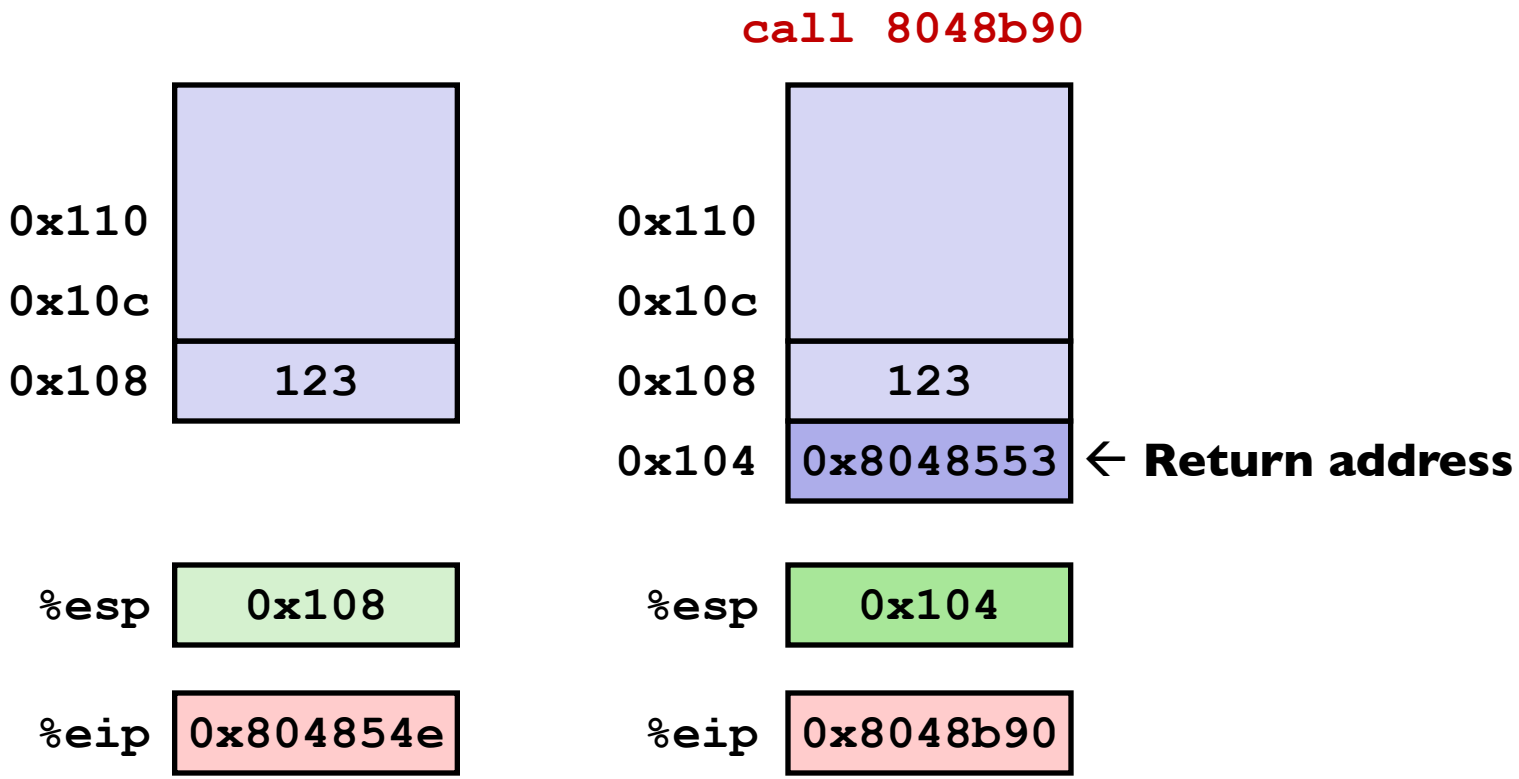
- **Procedure call: `call label`**
  - Push return address on stack
  - Jump to label

- **Return address**
  - Address of the next instruction right after call

- **Procedure return: `ret`**
  - Pop address from stack
  - Jump to address

Procedure call example

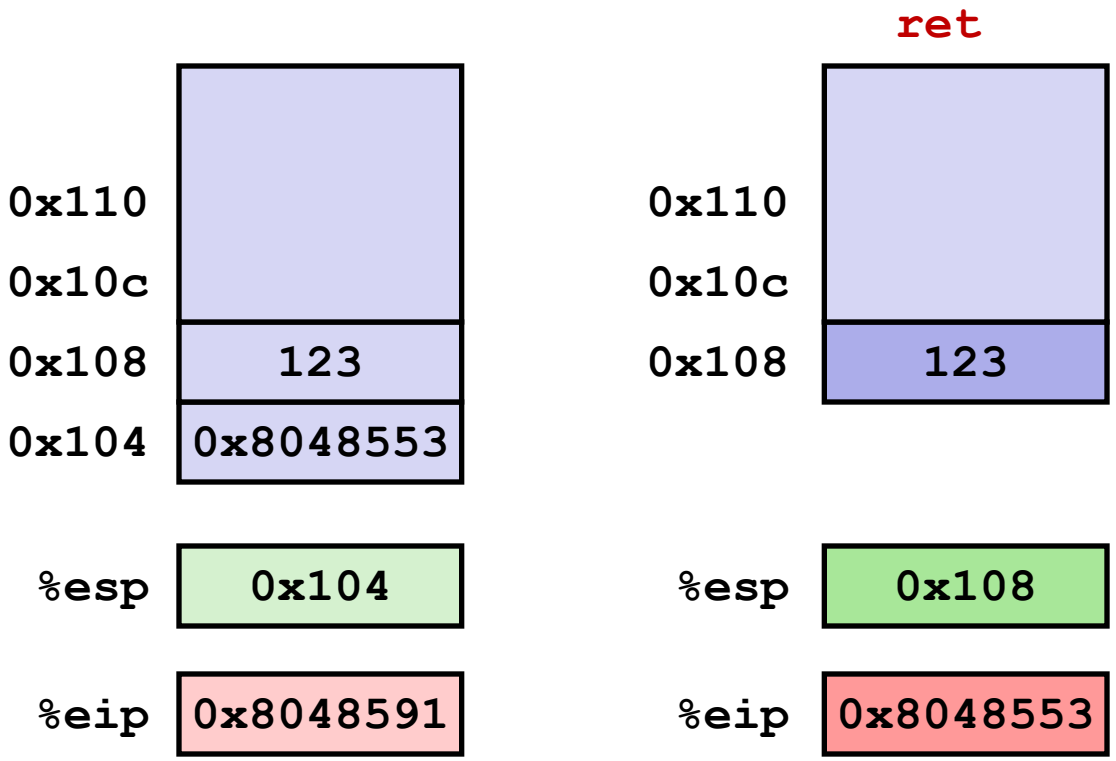
```
804854e:      e8 3d 06 00 00        call    8048b90 <main>
8048553:      50                    pushl   %eax
```



%eip: program counter

Procedure return example

```
8048591:    c3                    ret
```



%eip: program counter

Call chain

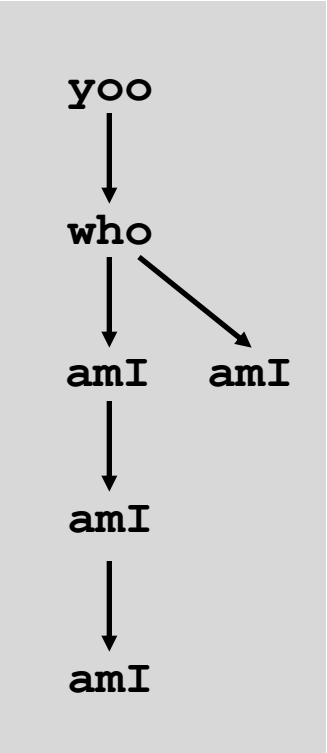
```
yoo(...)
{
   •
   •
   who();
   •
   •
}
```

```
who(...)
{
   • • •
   amI();
   • • •
   amI();
   • • •
}
```

```
amI(...)
{
   •
   •
   amI();
   •
   •
}
```

Procedure **amI()** is recursive

Call chain example

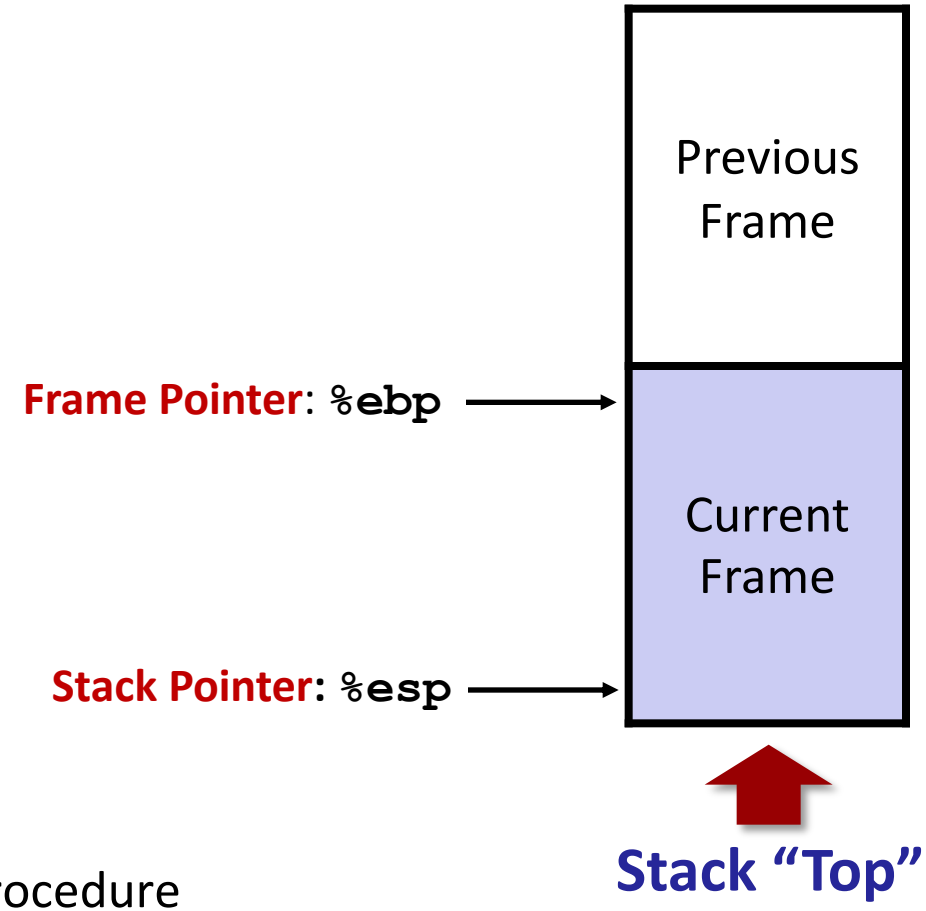


Call chain support

- **Need some place to store state of each instantiation**
  - Arguments
  - Local variables
  - Return address
- **Stack discipline**
  - State for each procedure needed only for limited time (until return)
  - Callee returns before caller does
- **Stack allocated in frames**
  - State for single procedure instantiation
  - Delimited by `%esp` and `%ebp`

Stack frames

- **Contents**
  - Local variables
  - Return information
  - Temporary space



- **Management**
  - Space allocated when enter procedure
    - "Set-up" code (**prologue**)
  - Deallocated when return
    - "Finish" code (**epilogue**)

Stack frames: Prologue and Epilogue

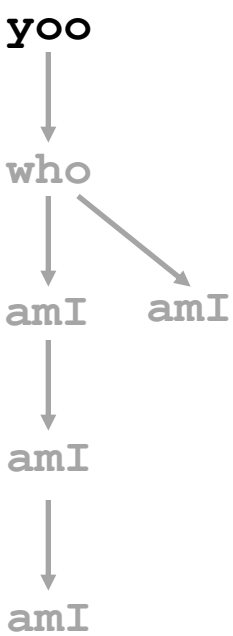
```
yoo:
  pushl %ebp          # Save old %ebp
  movl %esp,%ebp      # Set %ebp as frame pointer
  •
  •
  •
  call who
  •
  •
  •
  movl %ebp,%esp      # Set %esp to beginning of frame
  popl %ebp           # Restore saved %ebp
  ret
```

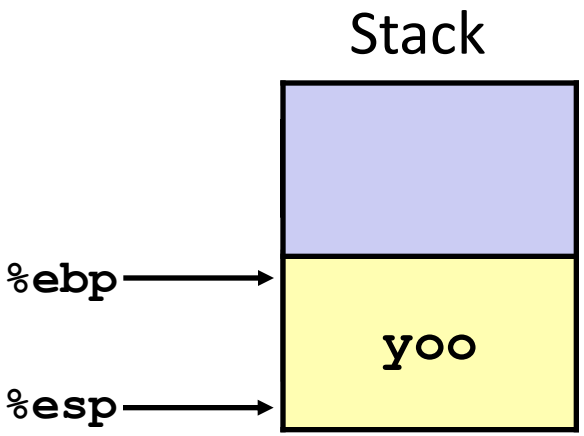
Prologue

Epilogue

Call chain example

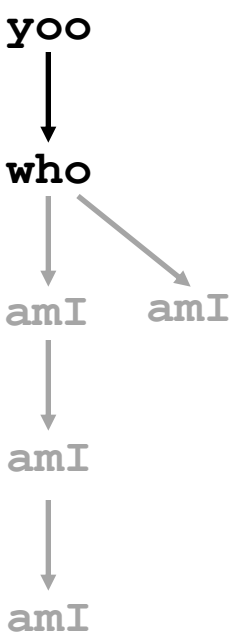
```
yoo(...)
{
    •
    •
    who();
    •
    •
}
```

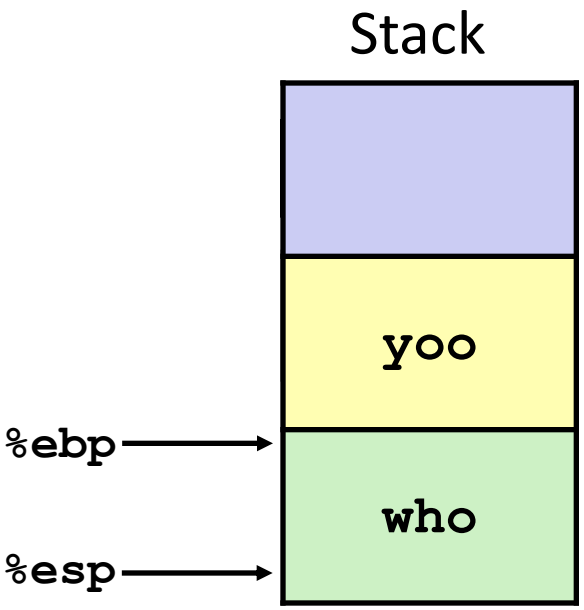




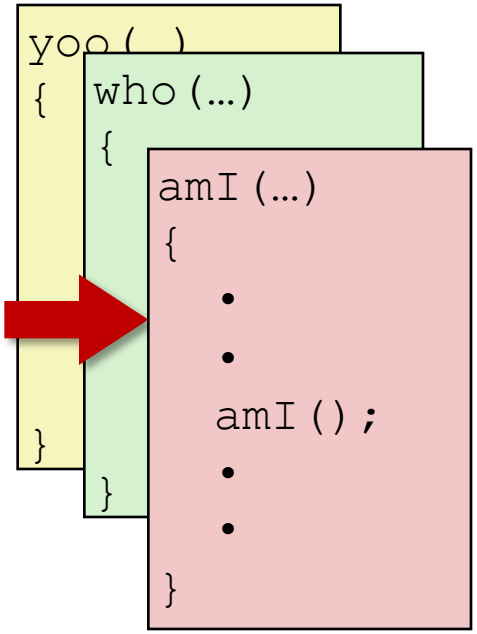
Call chain example

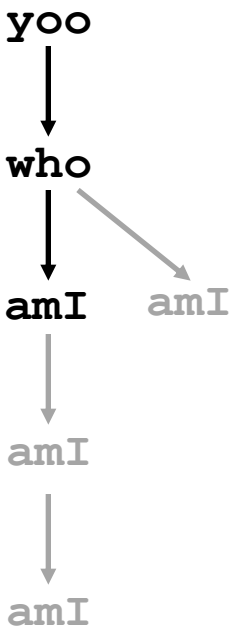
```
yoo( )
{
    who(…)
    {
        • • •
        amI();
        • • •
        amI();
        • • •
    }
}
```

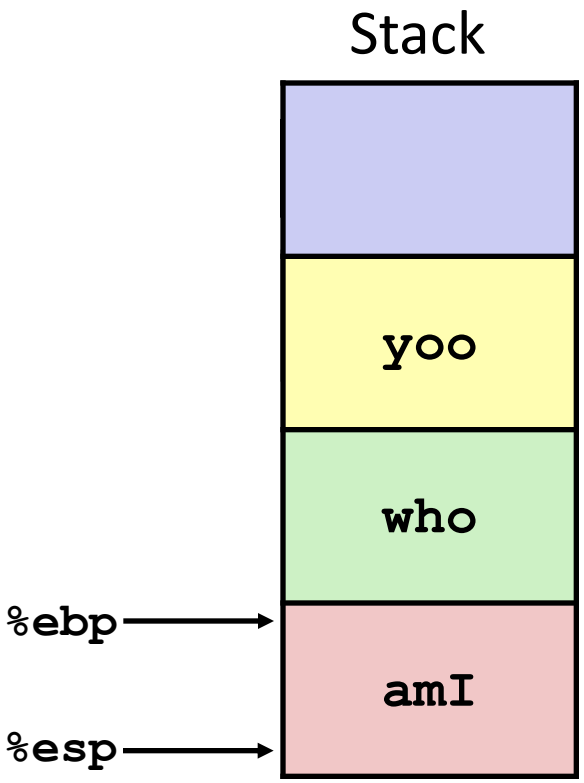




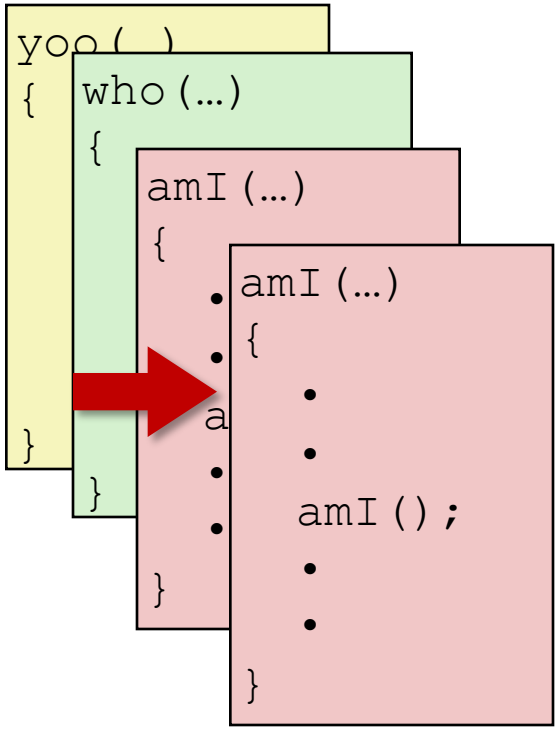
Call chain example

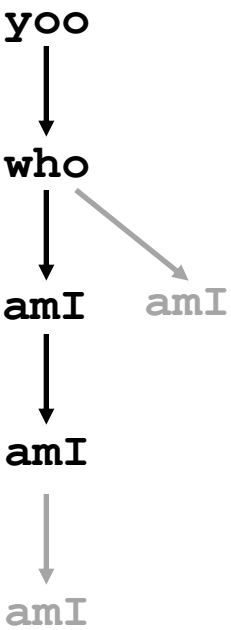


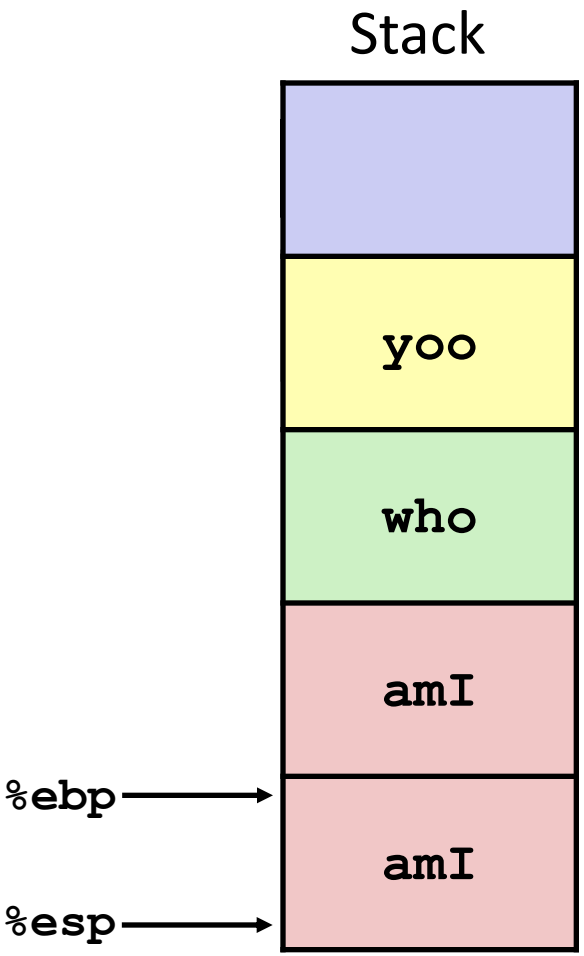




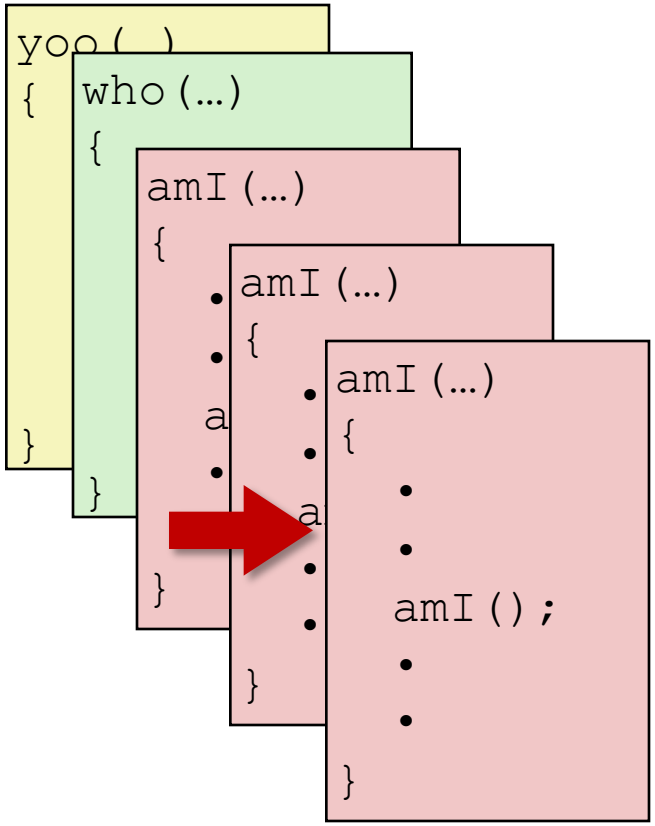
Call chain example

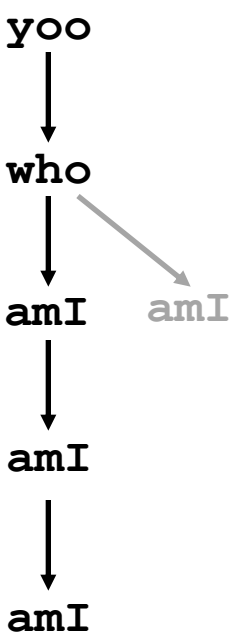


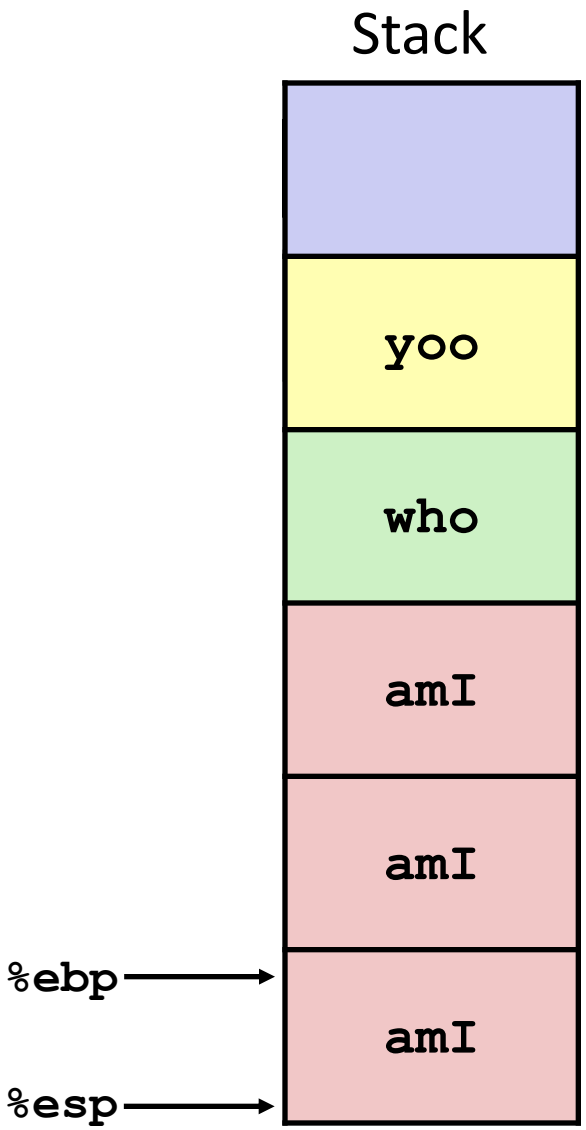




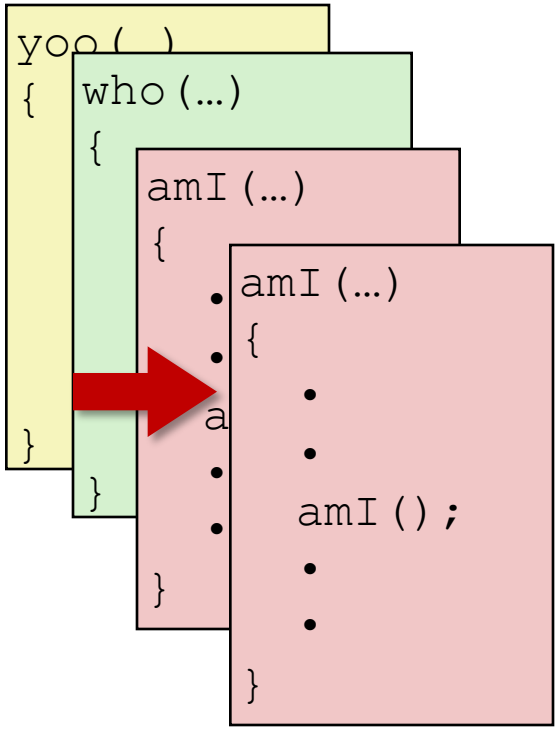
Call chain example

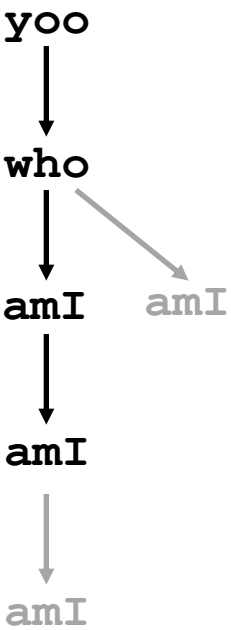


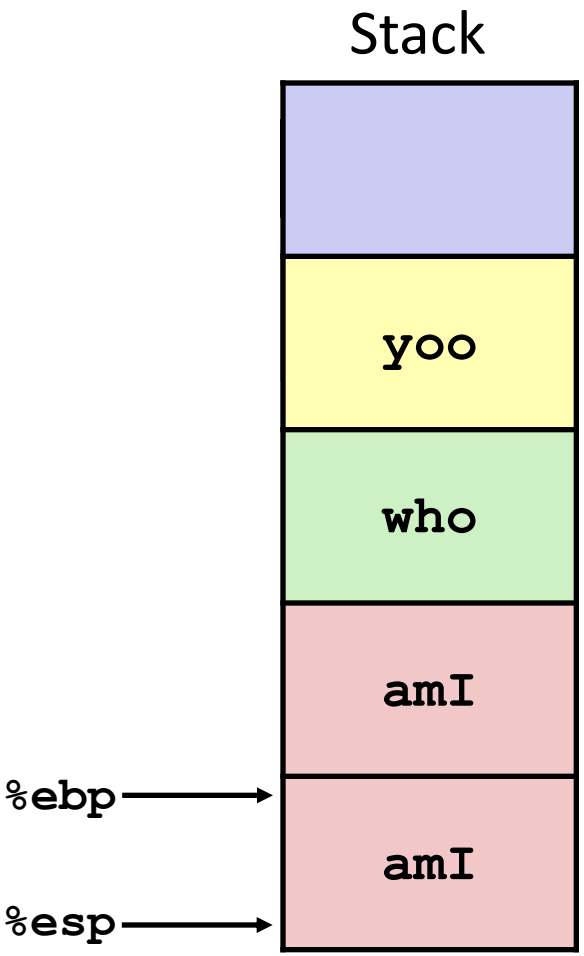




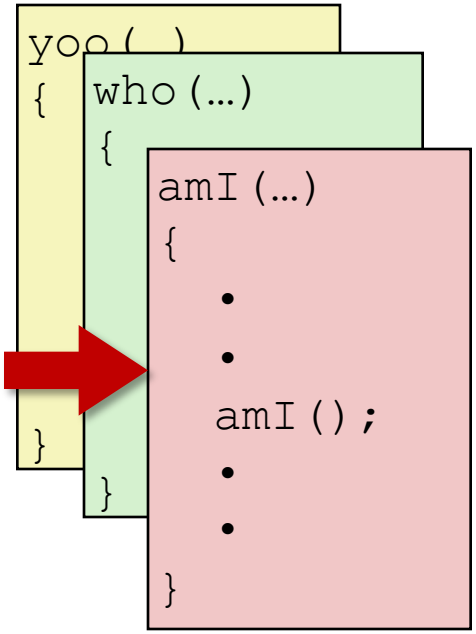
Call chain example

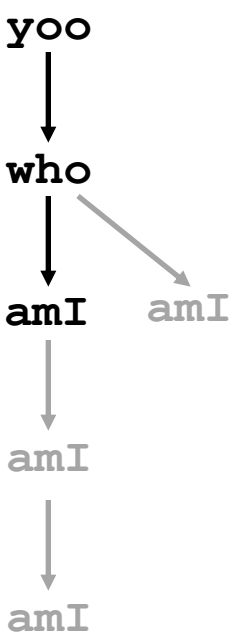


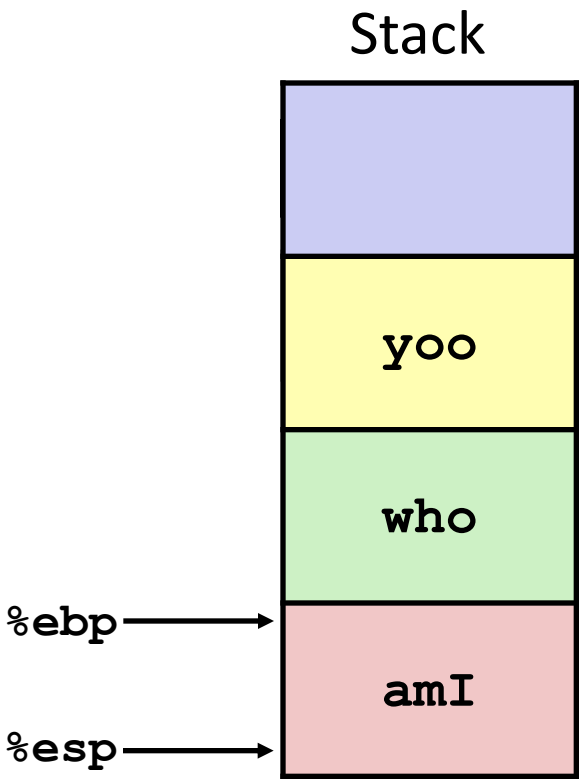




Call chain example

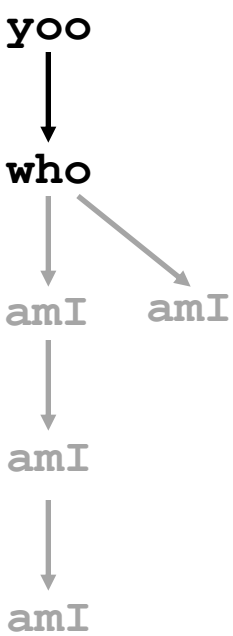


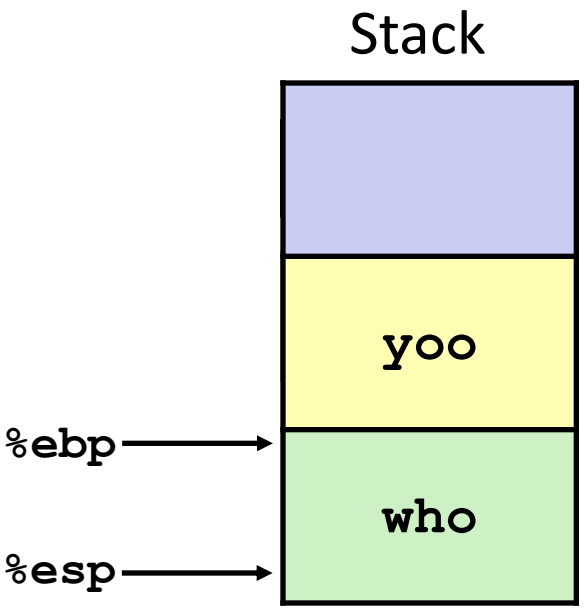




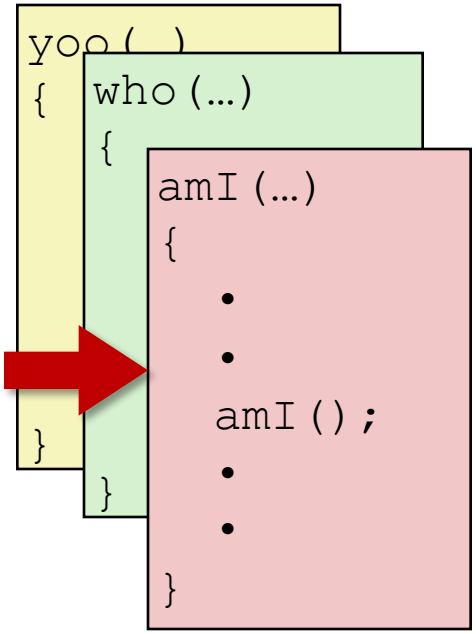
Call chain example

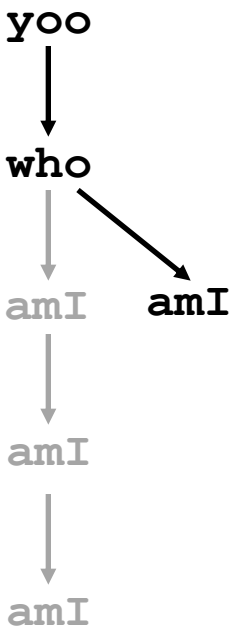
```
yoo( )
{
    who(…)
    {
        • • •
        amI();
        • • •
        amI();
        • • •
    }
}
```

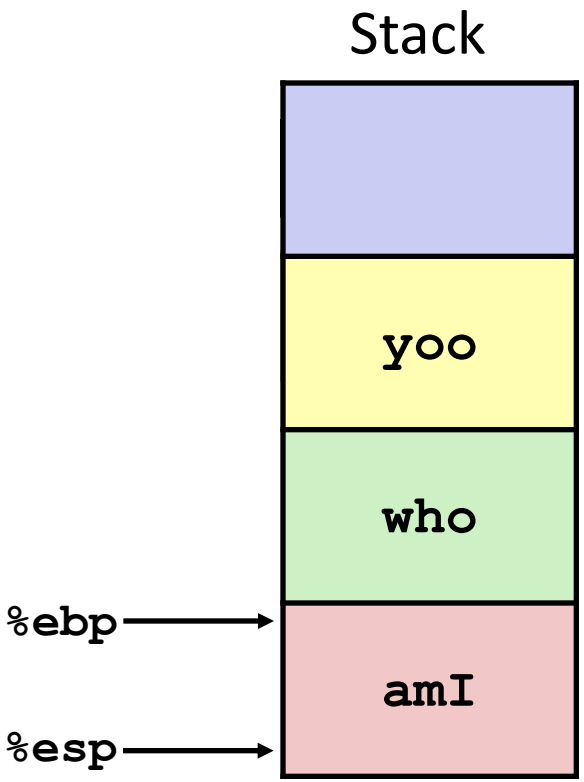




Call chain example

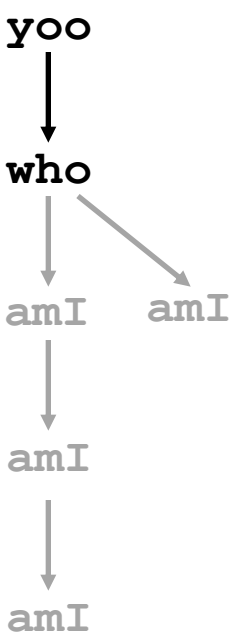


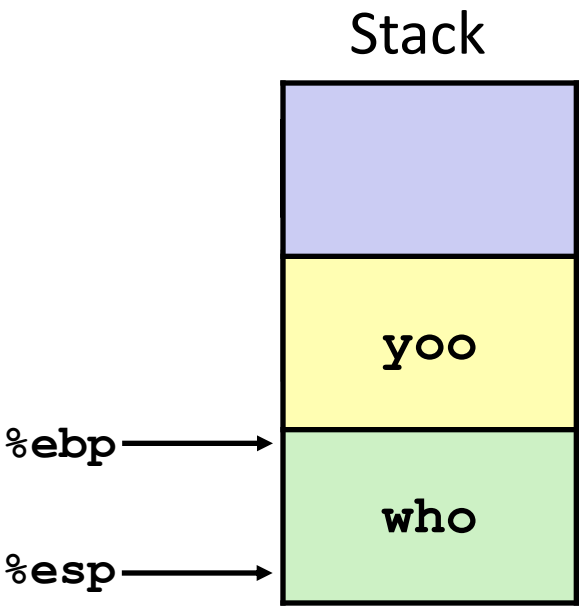




Call chain example

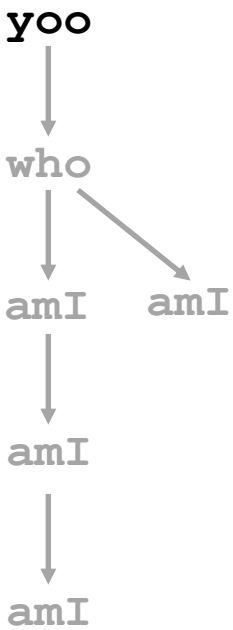
```
yoo( )
{
    who(…)
    {
        • • •
        amI();
        • • •
        amI();
        • • •
    }
}
```

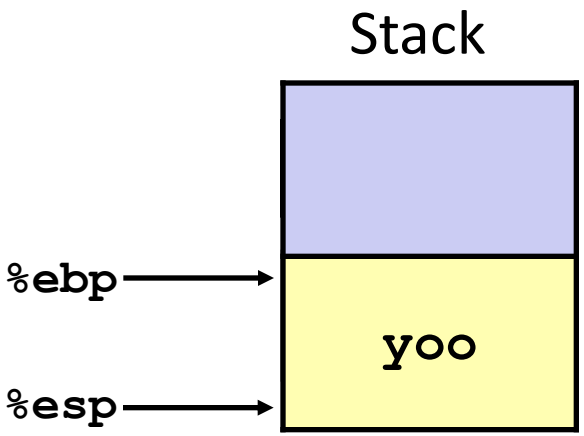




Call chain example

```
yoo(...)
{
    •
    •
    who();
    •
    •
}
```





Today

- Mechanisms in procedures
- Stack structure
- Passing control
- **Calling conventions**
- Passing data
- Managing local data

Register saving conventions

- **Can registers be safely used for temporary storage in both functions?**

```
yoo:
    …
    movl $15213, %edx
    call who
    addl %edx, %eax
    …
    ret
```

```
who:
    …
    movl 8(%ebp), %edx
    addl $18243, %edx
    …
    ret
```

Register saving conventions

- **Can registers be safely used for temporary storage in both functions?**

```
yoo:
    …
    movl $15213, %edx
    call who
    addl %edx, %eax
    …
    ret
```

```
who:
    …
    movl 8(%ebp), %edx
    addl $18243, %edx
    …
    ret
```

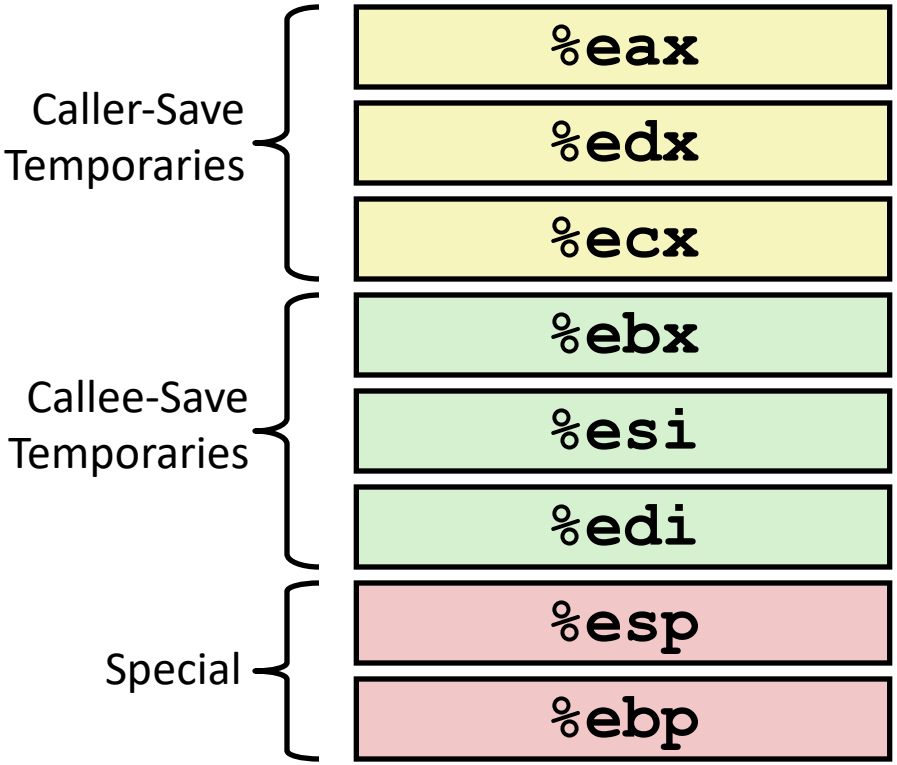
- **Contents of register `%edx` overwritten by `who`**

Register saving conventions

- **"Caller save"**
  - Caller saves temporary values in its frame before the call
- **"Callee save"**
  - Callee saves temporary values in its frame before using
- **Which registers are caller-save or callee-save?**

IA32 Register saving conventions

- **`%eax`, `%edx`, `%ecx`**
  - Caller saves prior to call if values are used later
- **`%ebx`, `%esi`, `%edi`**
  - Callee saves if wants to use them
- **`%esp`, `%ebp`**
  - Special form of callee save
  - Restored to original values upon exit from procedure



Register saving conventions

```
yoo:
    …
    movl $15213, %edx
    pushl %edx
    call who
    popl %edx
    addl %edx, %eax
    …
    ret
```

```
who:
    …
    movl 8(%ebp), %edx
    addl $18243, %edx
    …
    ret
```

- **Caller saves `%edx` before call and restores it after return**

Today

- Mechanisms in procedures
- **Stack structure**
- **Passing control**
- **Passing data**
- Calling conventions
- **Managing local data**

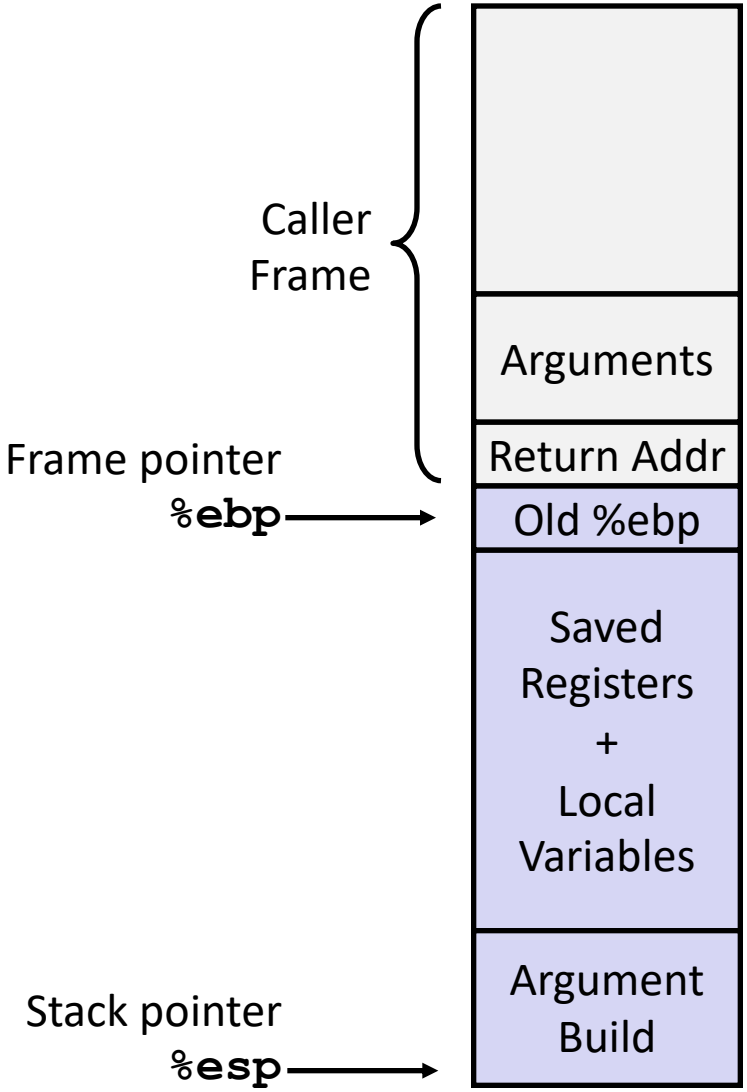
Passing data between procedures

- **A standard must be used to…**
  - Consistently place input parameters for functions to retrieve
  - Consistently place output values for the main program to retrieve
- **Most C compilers use the stack as a clean way to pass data to functions**
  - Without having to worry about clobbering registers or defining global variables
- **Likewise, the C style defines a common method for returning values**
  - Using the EAX register for 32-bit results and the EDX:EAX register pair for 64-bit integer values

IA32/Linux stack frame

- **Caller stack frame**
  - Arguments for the call
  - Return address (pushed by **`call`**)

- **Current stack frame**
  - Old frame pointer
  - Local variables (if used)
  - Saved register context
  - "Argument build:"
    Parameters for function about to call



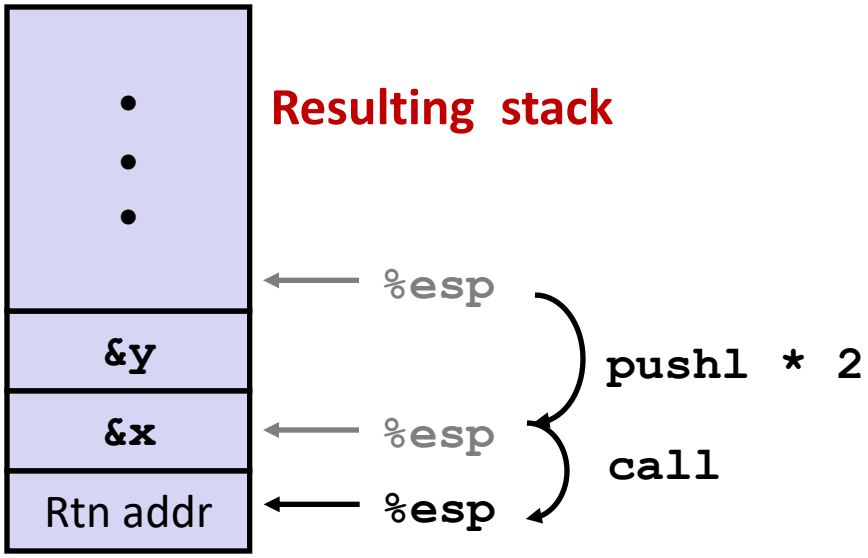
Revisiting swap

```
int x = 15213;
int y = 18243;

void call_swap() {
  swap(&x, &y);
}
```

```
call_swap:
    . . .
    pushl   $y
    pushl   $x
    call    swap
    . . .
```

```
void swap(int *xp, int *yp)
{
  int t0 = *xp;
  int t1 = *yp;
  *xp = t1;
  *yp = t0;
}
```



Revisiting swap

```
void swap(int *xp, int *yp)
{
  int t0 = *xp;
  int t1 = *yp;
  *xp = t1;
  *yp = t0;
}
```

```
swap:
   pushl %ebp
   movl  %esp, %ebp
   pushl %ebx

   movl  8(%ebp), %edx
   movl  12(%ebp), %ecx
   movl  (%edx), %ebx
   movl  (%ecx), %eax
   movl  %eax, (%edx)
   movl  %ebx, (%ecx)

   popl  %ebx
   movl  %esp, %ebp
   popl  %ebp
   ret
```

Set Up

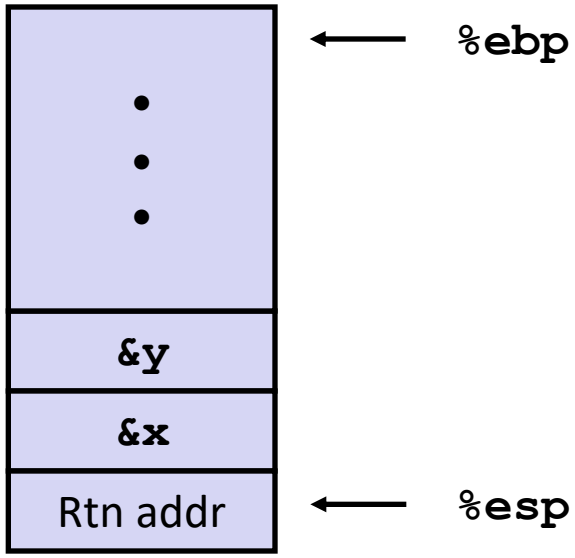
Body

Finish

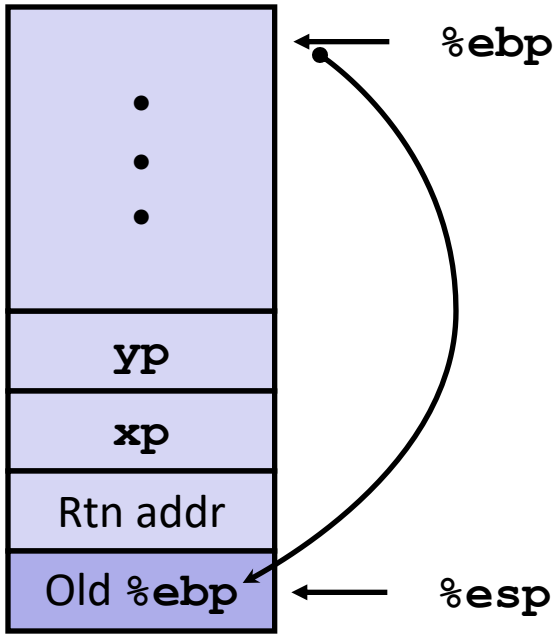
- **Observation**
  - Saved and restored register **`%ebx`**

swap setup

Entering stack



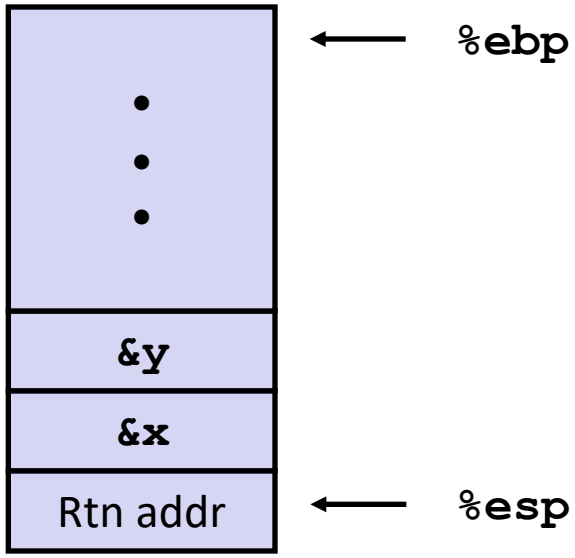
Resulting stack



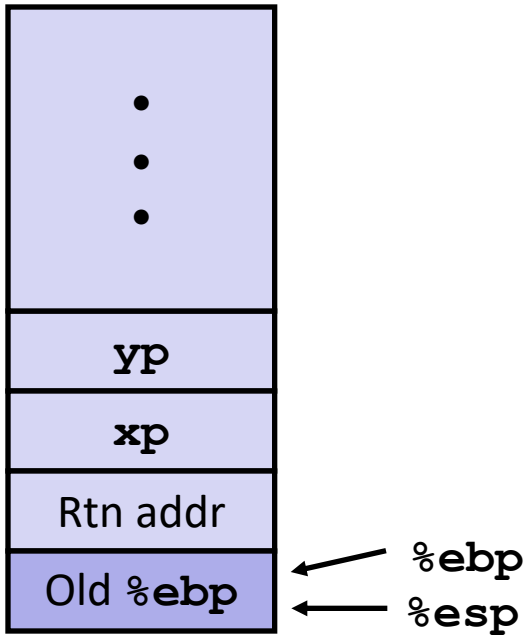

```
swap:
    pushl %ebp
    movl %esp,%ebp
    pushl %ebx
```

swap setup

Entering stack



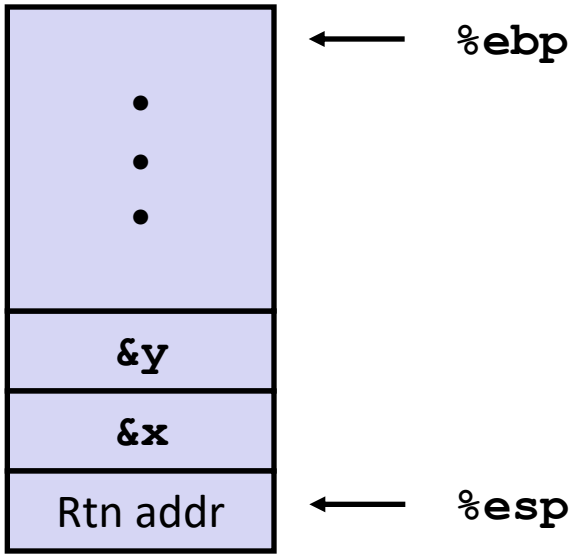
Resulting stack



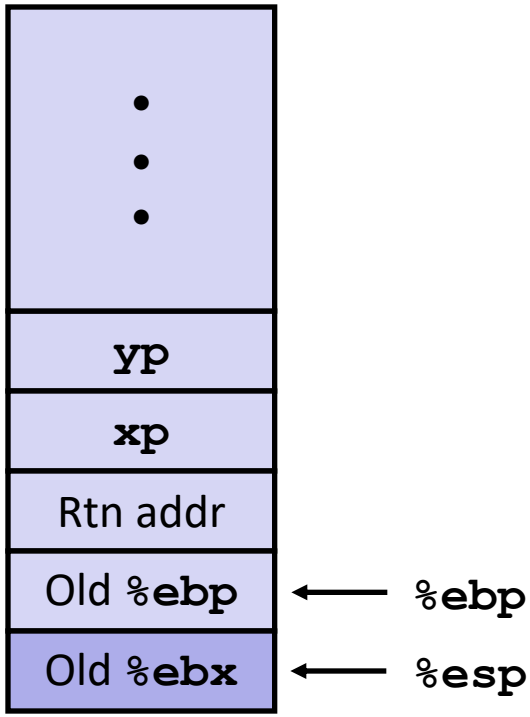

```
swap:
    pushl %ebp
    movl %esp,%ebp
    pushl %ebx
```

swap setup

Entering stack

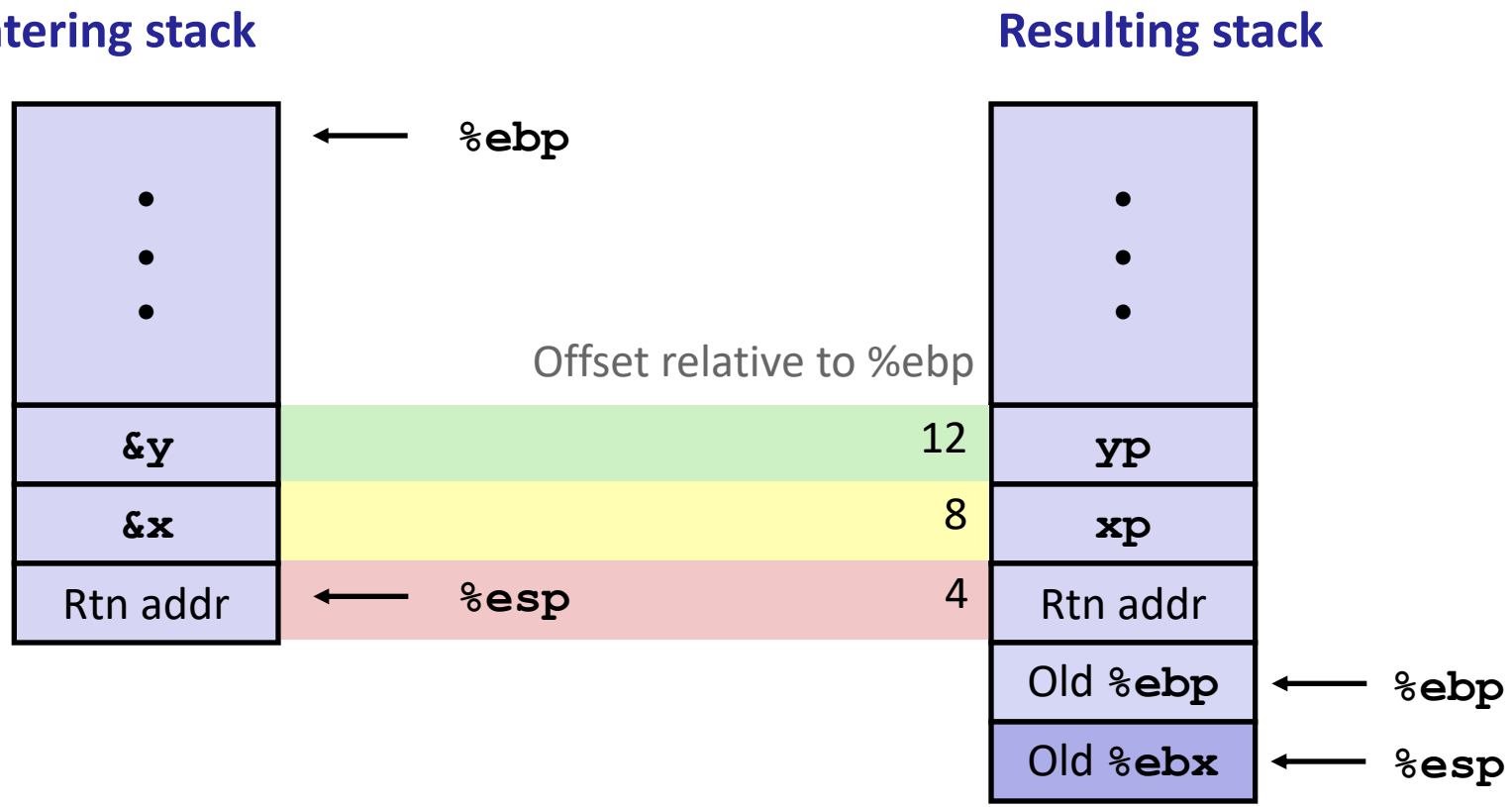


Resulting stack




```
swap:
    pushl %ebp
    movl %esp,%ebp
    pushl %ebx
```

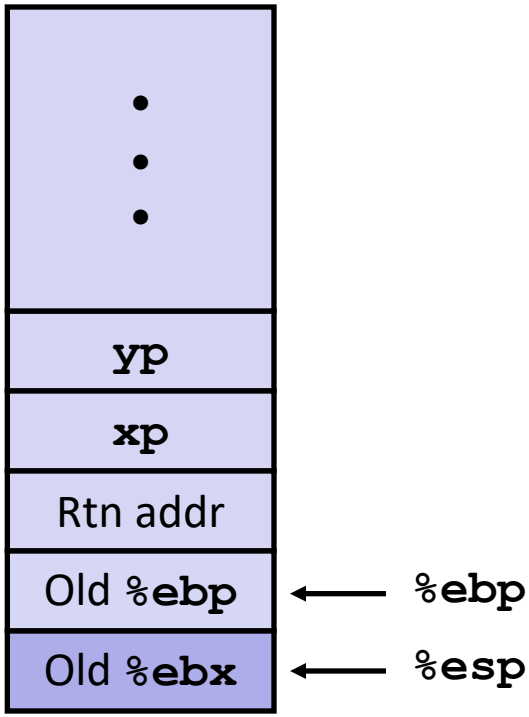
swap body



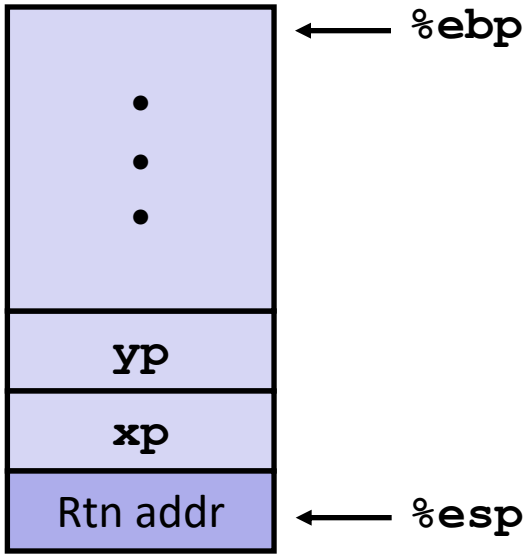

```
movl 8(%ebp),%edx    # get xp
movl 12(%ebp),%ecx   # get yp
movl (%edx), %ebx
…
```

swap finish

Stack before finish



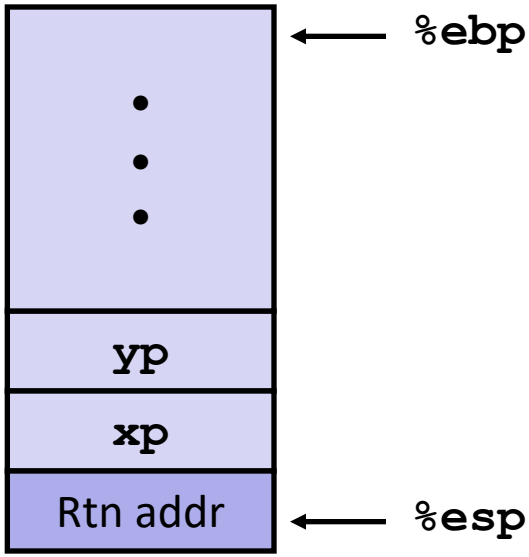
Resulting stack



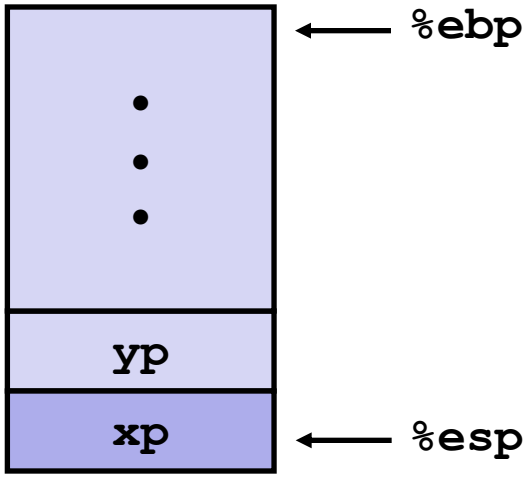

```
popl %ebx
movl %ebp, %esp
popl %ebp
…
```

swap return

Stack before return



Resulting stack




```
…
ret
```

Cleaning out the stack

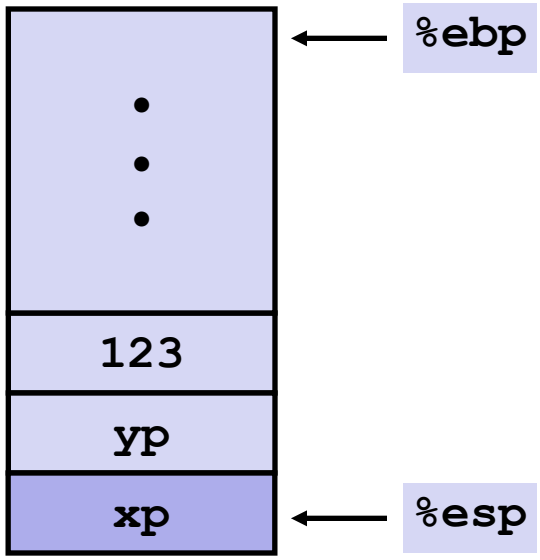
- **Caller places all input values on stack before call**

- **When the callee returns, those values are still on the stack**
  - The function accessed them without popping them off of the stack

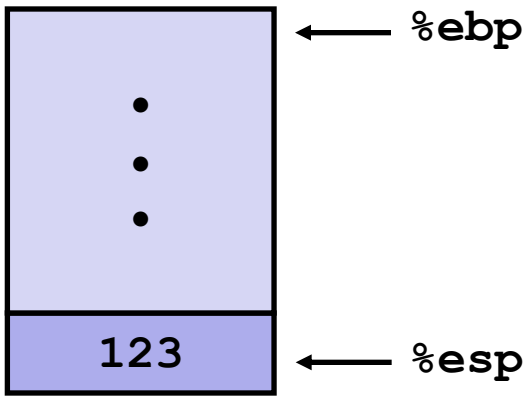
- **Cleaning can be performed by:**
  1. Using POP to retrieve each parameter in turn
  2. Move `%esp` back to the original location before the function call
     - Can be done in a single operation by adding the size of the input elements

Cleaning out the stack

Stack after return



Resulting stack




```
call swap
addl $8, %esp
…
```

Disassembled swap

```
08048384 <swap>:
 8048384:  55                        pushl   %ebp
 8048385:  89 e5                     movl    %esp,%ebp
 8048387:  53                        pushl   %ebx
 8048388:  8b 55 08                  movl    0x8(%ebp),%edx
 804838b:  8b 4d 0c                  movl    0xc(%ebp),%ecx
 804838e:  8b 1a                     movl    (%edx),%ebx
 8048390:  8b 01                     movl    (%ecx),%eax
 8048392:  89 02                     movl    %eax,(%edx)
 8048394:  89 19                     movl    %ebx,(%ecx)
 8048396:  5b                        popl    %ebx
 8048397:  5d                        popl    %ebp
 8048398:  c3                        ret
```

Calling code

```
 80483b4:  pushl  $y                      # Copy &y
 80483bc:  pushl  $x                      # Copy &x
 80483c3:  call   8048384 <swap>          # Call swap
 80483c8:  addl   $8, %esp                # Clean swap params
 80483c9:  leave                          # Prepare to return
 80483d0:  ret                            # Return
```

Today

- Mechanisms in procedures
- Stack structure
- Passing control
- Passing data
- Calling conventions
- **Managing local data**

Managing local data

- **When looking for an easy location for working storage for data elements within the function, the stack again comes to the rescue**

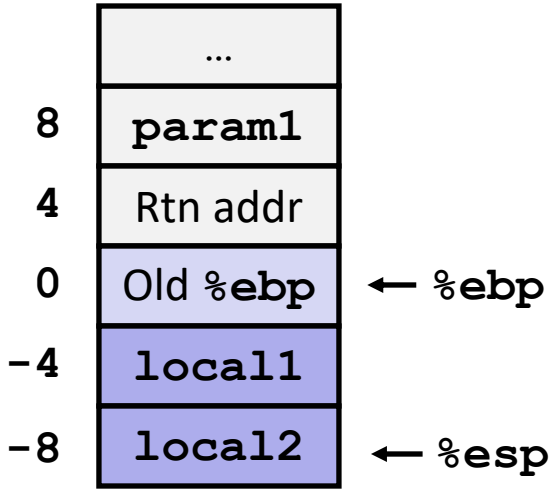
- **After function setup, `%ebp` points to the top of the stack**
  - Any additional data used in the function can be placed on the stack after that point without affecting how the input values are accessed

- **Local variables are also referenced using `%ebp`**
  - Assuming 4-byte data values, the first local variable can be accessed by referencing `-4(%ebp)`, while the second local variable can be accessed by referencing `-8(%ebp)`, …

Managing local data

- **You must remember to reserve enough space for all of the local variables needed in the function**
  - Include one additional line in function setup to reserve the space for the local variables by moving **`%esp`** down

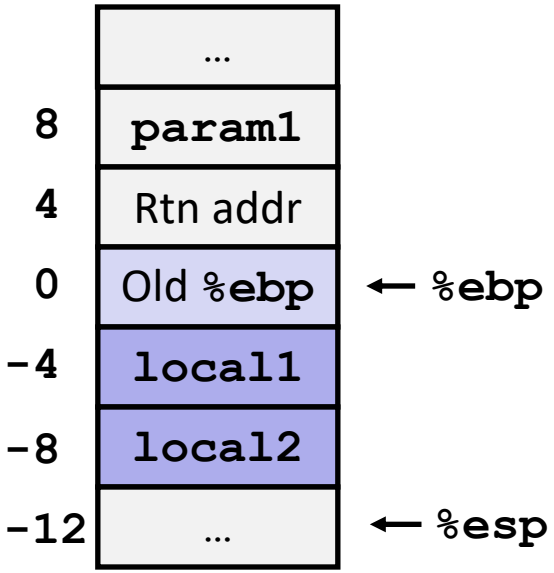
```
function:
    pushl %ebp
    movl %ebp, %esp
    subl $8, %esp
    …
```



- **This code reserves 8 bytes to be used for local variables**
  - This space can be used, for e.g., as either two 32 bit values or one 64 bit value

Managing local data

- **If any data is now pushed onto the stack**
  - It will be placed beneath the local variables, preserving them so they can still be accessed via %ebp



- **When the end of the function is reached `%esp` is set back to its original value**
  - The local variables will be lost from the stack
  - They will not be directly accessible using `%esp` or `%ebp` from the calling function (thus the term *local variables*)

Managing local data: Example

- **Function `add3` creates a pointer to a local variable and passes it to function `incrk`**

Generating pointer

```
/* Compute x + 3 */
int add3(int x){
  int localx = x;
  incrk(&localx, 3);
  return localx;
}
```

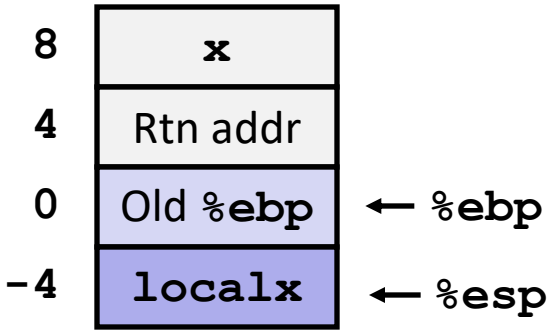
Referencing pointer

```
/* Increment value by k */
void incrk(int *ip, int k){
  *ip += k;
}
```

Creating and initializing the local variable

```
int add3(int x) {
  int localx = x;
  incrk(&localx, 3);
  return localx;
}
```

- Variable `localx` must be stored on stack
- Compute pointer as -4(%ebp)



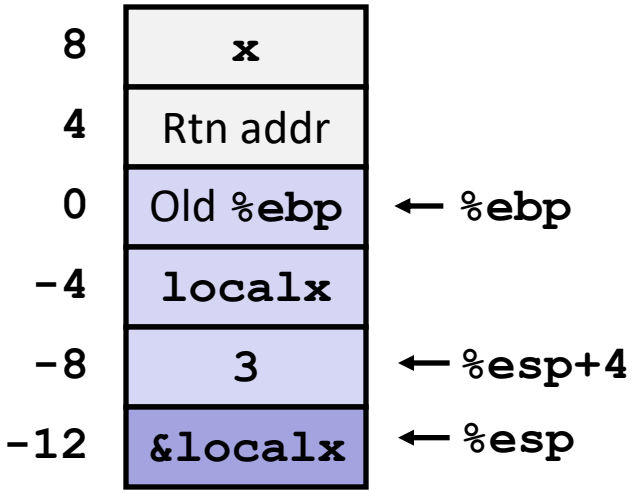
First part of add3

```
add3:
   pushl %ebp
   movl  %esp,%ebp
   subl  $4,%esp          # Allocate 4 bytes
   movl  8(%ebp),%eax
   movl  %eax,-4(%ebp)    # Set localx to x
```

Creating pointer as argument

```
int add3(int x) {
  int localx = x;
  incrk(&localx, 3);
  return localx;
}
```

- **Use leal instruction to compute address of `localx`**



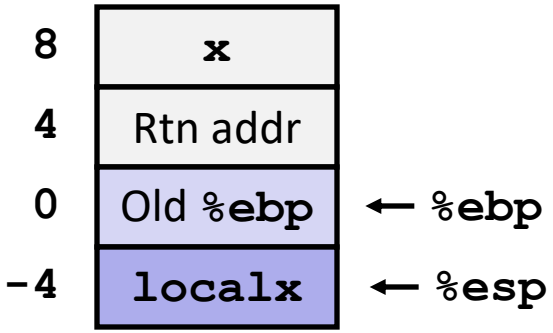
Middle part of add3

```
pushl $3                # 2nd arg = 3
leal -4(%ebp),%eax      # %eax = &localx
pushl %eax              # 1st arg = &localx
call incrk
```

Retrieving local variable

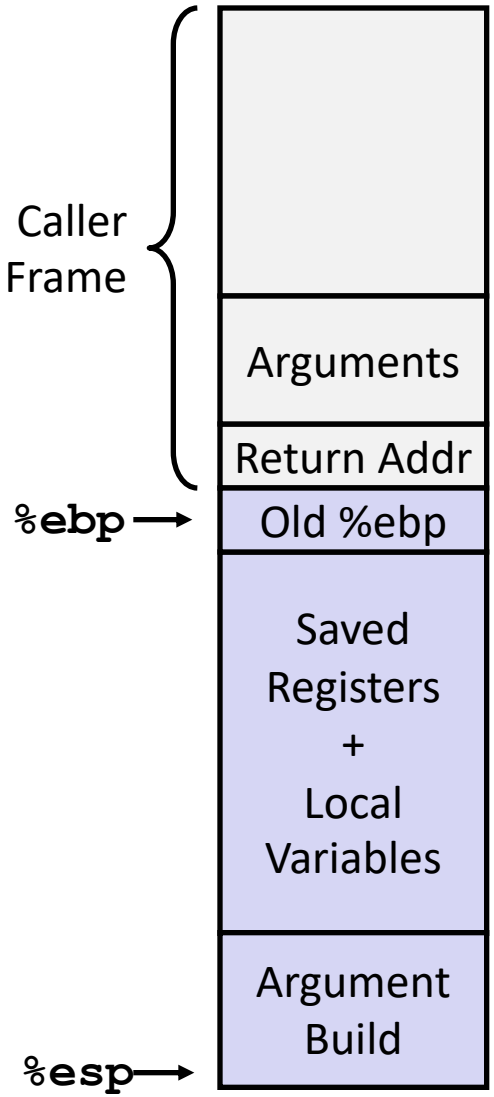
```
int add3(int x) {
  int localx = x;
  incrk(&localx, 3);
  return localx;
}
```

- **Retrieve `localx` from stack as return value**



Final part of add3

```
  movl  -4(%ebp),%eax     # Return localx
  leave
  ret
```

IA32 Procedure summary

- **Stack is the right data structure for**
  - Passing control
  - Passing data
  - Managing local data
- **Calling conventions**
  - Can safely store values in local stack frame and in callee-saved registers
  - Put function arguments at top of stack
  - Result return in `%eax`
- **Pointers are addresses of values**
  - On stack or global